Högskolan Kristianstad
291 88 Kristianstad
044-250 30 00
www.hkr.se

Självständigt arbete (examensarbete), 15 hp, för (formatmall Normal)
examensarbete ex: Kandidatexamen i pedagogik
Termin år ex: VT 2021
Fakulteten för …

Fotball Training tracker

Oskar Mosleh

Abstract

This report presents the development of a full stack web application called Football Training Tracker. The purpose of the system is to manage football players, training sessions, and attendance in a simple and structured way.

The application is built using React for the frontend, Node.js with Express for the backend, and MongoDB Atlas as the database. The system is based on three main collections, which are Player, Session, and Attendance. These are connected using references to allow data to be shared between different parts of the application.

The user can create players, create training sessions, assign players to sessions, and register attendance. The frontend communicates with the backend through a REST API, and all data is stored in the database.

The project shows how a full stack system can be built from start to finish and how different technologies work together. It also helped improve understanding of database design and how to handle relationships in a NoSQL database.

Innehållsförteckning

Abstract **2**

1. Introduction **4**

2. System Overview 4

3. Database Design *4*

4. API and Implementation *5*

5. Frontend Implementation *6*

6. Result *6*

7. Discussion *6*

8. Conclusion *7*

1. Introduction

This project describes the development of a full stack web application for managing football training sessions. The main idea behind the system is to create a simple tool where a coach can organize players, create training sessions, and track attendance in one place. In many cases, this type of information is handled manually, which can be inefficient and difficult to manage over time.

The goal of the project was to build a working application and at the same time understand how frontend, backend, and database systems work together. A strong focus was placed on learning how data moves between different parts of the system and how to structure that data in a clear way.

2. System Overview

The application is built using a client server architecture. The frontend is developed using React and is responsible for everything the user sees and interacts with. The backend is built using Node.js and Express, and it handles all logic, data processing, and communication with the database. MongoDB Atlas is used to store all data online.

The system allows the user to create players, create training sessions, and connect players to those sessions. It also includes a way to track attendance by connecting players and sessions through a separate data model. The frontend sends requests to the backend using HTTP, and the backend responds with data in JSON format.

3. Database Design

The database is designed using three main collections called Player, Session and Attendance. The Player collection stores information about each player, such as name, age, and position. The Session collection represents a training session and includes information like title, date, focus area, and intensity level. It also contains references to players, which makes it possible to connect multiple players to a session.

The Attendance collection is used to track which players attended a specific session. It connects a player and a session using references and also stores additional information such as attendance status, rating, and notes. This makes it possible to store more detailed data about each training session.

The relationship between these collections is handled using ObjectId references. This approach makes it possible to connect data between collections and retrieve related information when needed. For example, when a session is requested, the system can also return the players connected to that session.

4. API and Implementation

The backend is structured in a clear way using routes, controllers, and models. Each part has a specific responsibility, which makes the code easier to understand and maintain. Routes define the available endpoints, controllers contain the logic, and models define how the data is stored in the database.

The system supports create, read, update, and delete operations for the main parts of the application. When a new player is created, the frontend sends data to the backend, which then saves it in the database and returns the created object. When sessions are requested, the backend returns all sessions along with the players connected to them.

The backend also includes basic error handling to make sure that incorrect requests do not break the system. This structure makes the application easier to expand in the future.

One example of an API endpoint is the route used to create a player. A POST request is sent to the server with data such as name, age, and position. The server then validates the data and stores it in the database before returning the created player.

Another example is the endpoint used to get all sessions. A GET request returns all sessions together with their connected players. This allows the frontend to display full information about each session.

5. Frontend Implementation

The frontend is built using React and is divided into smaller components to keep the structure clean. Each component is responsible for a specific part of the application. One component is used for managing players, another for creating sessions, and another for displaying sessions.

The user interacts with the system through forms and buttons. When a user performs an action, such as creating a player or a session, the frontend sends a request to the backend. When the data is returned, the interface updates automatically to show the new information.

This structure makes the application easier to understand and improves the overall user experience.

6. Result

The final result is a working full stack application that allows the user to manage football training in a simple way. The user can create players, create sessions, assign players to sessions, and track attendance.

All data is stored in the database and displayed in the frontend. The connection between frontend and backend works as expected, and all main features are implemented.

7. Discussion

One of the main challenges during this project was understanding how to work with relationships in MongoDB. Since MongoDB is not a relational database, it required a different way of thinking compared to traditional SQL databases. This was solved by using ObjectId references to connect collections.

Another challenge was making sure that data stays consistent across the system. For example, when a player is deleted, the system must handle that change without causing problems in sessions or attendance records.

The project helped build a better understanding of how a full stack application is structured and how different technologies work together in practice.

8. Conclusion

This project resulted in a complete full stack application that meets the requirements of the assignment. The system demonstrates how a real problem can be solved using modern web technologies.

The work has improved knowledge in both frontend and backend development, as well as database design. It has also provided practical experience in building and connecting all parts of a system from start to finish.